

Node.js 교과서



Node.js 교과서 개정2판

길벗

11장

11.1 테스트 준비하기

11.2 유닛테스트

11.3 테스트 커버리지

11.4 통합 테스트

11.5 부하 테스트

11.1 테스트 준비하기

1. 테스트를 하는 이유

» 자신이 만든 서비스가 제대로 동작하는지 테스트해야 함

- 기능이 많다면 수작업으로 테스트하기 힘들
- 프로그램이 프로그램을 테스트할 수 있도록 자동화함
- 테스트 환경을 최대한 실제 환경과 비슷하게 흉내냄
- 아무리 철저하게 테스트해도 에러를 완전히 막을 수는 없음

» 테스트를 하면 좋은 점

- 하지만 허무한 에러로 인해 프로그램이 고장나는 것은 막을 수 있음
- 한 번 발생한 에러는 테스트로 만들어두면 같은 에러가 발생하지 않게 막을 수 있음
- 코드를 수정할 때 프로그램이 자동으로 어떤 부분이 고장나는 지 알려줌



2. Jest 설치하기

» npm i -D jest

- Nodebird 프로젝트를 그대로 사용함

package.json

```
{
  "name": "nodebird",
  "version": "0.0.1",
  "description": "익스프레스로 만드는 SNS 서비스",
  "main": "app.js",
  "scripts": {
    "start": "nodemon app",
    "test": "jest"
  },
  ...
}
```

3. 테스트 실행해보기

» routes 폴더 안에 middlewares.test.js 작성

- 테스트용 파일은 파일명에 test나 spec이 있으면 됨
- npm test로 test나 spec 파일들을 테스트함.
- 테스트를 아무것도 작성하지 않았으므로 에러 발생(테스트 실패)

콘솔

```
$ npm test
> nodebird@0.0.1 test C:\Users\zerocho\nodebird
> jest
```

FAIL routes/middlewares.test.js

- Test suite failed to run

Your test suite must contain at least one test.

```
Test Suites: 1 failed, 1 total
Tests:       0 total
Snapshots:   0 total
Time:        3.038s
Ran all test suites.
npm ERR! Test failed.  See above for more details.
```



4. 첫 테스트 코드 작성하기

» middlewares.test.js 작성하기

- test 함수의 첫 번째 인수로 테스트에 대한 설명
- 두 번째 인수인 함수에는 테스트 내용을 적음
- expect 함수의 인수로 실제 코드를, toEqual 함수의 인수로는 예상되는 결과값을

routes/middlewares.test.js

```
test('1 + 1은 2입니다.', () => {  
  expect(1 + 1).toEqual(2);  
});
```

- expect와 toEqual의 인수가 일치하면 테스트 통과

콘솔

```
$ npm test  
> nodebird@0.0.1 test C:\Users\zerocho\nodebird  
> jest
```

```
PASS routes/middlewares.test.js  
✓ 1 + 1은 2입니다. (3ms)
```

```
Test Suites: 1 passed, 1 total  
Tests:      1 passed, 1 total  
Snapshots:  0 total  
Time:       2.583s  
Ran all test suites.
```



5. 실패하는 경우

» 두 인수를 다르게 작성하면 실패(메시지를 살펴볼 것)

routes/middlewares.test.js

```
test('1 + 1은 2입니다.', () => {  
  expect(1 + 1).toEqual(3);  
});
```

콘솔

```
$ npm test  
> nodebird@0.0.1 test C:\Users\zerocho\nodebird  
> jest
```

FAIL routes/middlewares.test.js

× 1 + 1은 2입니다. (4ms)

● 1 + 1은 2입니다.

expect(received).toEqual(expected) // deep equality

Expected: 3

Received: 2

```
1 | test('1 + 1은 2입니다.', () => {  
> 2 |   expect(1 + 1).toEqual(3);  
    |                   ^  
3 | });  
4 |
```

at Object.<anonymous> (routes/middlewares.test.js:2:17)

Test Suites: 1 failed, 1 total

Tests: 1 failed, 1 total

Snapshots: 0 total

Time:

2.657s

Ran all test suites.

npm ERR! Test failed. See above for more details.

11.2 유닛 테스트



1. middlewares 테스트하기

» middlewares.test.js 작성하기

- 테스트 틀 잡기
- describe로 테스트 그룹화 가능

routes/middlewares.test.js

```
const { isLoggedIn, isNotLoggedIn } = require('./middlewares');

describe('isLoggedIn', () => {
  test('로그인되어 있으면 isLoggedIn이 next를 호출해야 함', () => {
  });

  test('로그인되어 있지 않으면 isLoggedIn이 에러를 응답해야 함', () => {
  });
});

describe('isNotLoggedIn', () => {
  test('로그인되어 있으면 isNotLoggedIn이 에러를 응답해야 함', () => {
  });

  test('로그인되어 있지 않으면 isNotLoggedIn이 next를 호출해야 함', () => {
  });
});
```



2. req, res 모킹하기

» 미들웨어 테스트를 위해 req와 res를 가짜로 만들어주어야 함

- jest.fn으로 함수 모킹 가능

routes/middlewares.test.js

```
const { isLoggedIn, isNotLoggedIn } = require('./middlewares');

describe('isLoggedIn', () => {
  const res = {
    status: jest.fn(() => res),
    send: jest.fn(),
  };
  const next = jest.fn();

  test('로그인되어 있으면 isLoggedIn이 next를 호출해야 함', () => {
    const req = {
      isAuthenticated: jest.fn(() => true),
    };
    isLoggedIn(req, res, next);
    expect(next).toHaveBeenCalledTimes(1);
  });

  test('로그인되어 있지 않으면 isLoggedIn이 에러를 응답해야 함', () => {
    const req = {
      isAuthenticated: jest.fn(() => false),
    };
    isLoggedIn(req, res, next);
    expect(res.status).toHaveBeenCalledWith(403);
    expect(res.send).toHaveBeenCalledWith('로그인 필요');
  });
});
```

콘솔

```
$ npm test
> nodebird@0.0.1 test C:\Users\zerocho\nodebird
> jest

PASS routes/middlewares.test.js
  isLoggedIn
    ✓ 로그인되어 있으면 isLoggedIn이 next를 호출해야 함 (3ms)
    ✓ 로그인되어 있지 않으면 isLoggedIn이 에러를 응답해야 함 (2ms)
  isNotLoggedIn
    ✓ 로그인되어 있으면 isNotLoggedIn이 에러를 응답해야 함
    ✓ 로그인되어 있지 않으면 isNotLoggedIn이 next를 호출해야 함 (1ms)

Test Suites: 1 passed, 1 total
Tests:      4 passed, 4 total
Snapshots:  0 total
Time:       2.689s
Ran all test suites.
```



3. expect 메서드

» expect에는 toEqual 말고도 많은 메서드 지원

- toBeCalledWith로 인수 체크
- toBeCalledTimes로 호출 회수 체크

routes/middlewares.test.js

```
...
describe('isNotLoggedIn', () => {
  const res = {
    redirect: jest.fn(),
  };
  const next = jest.fn();

  test('로그인되어 있으면 isNotLoggedIn이 에러를 응답해야 함', () => {
    const req = {
      isAuthenticated: jest.fn(() => true),
    };
    isNotLoggedIn(req, res, next);
    const message = encodeURIComponent('로그인한 상태입니다. ');
    expect(res.redirect).toBeCalledWith(`/?error=${message}`);
  });

  test('로그인되어 있지 않으면 isNotLoggedIn이 next를 호출해야 함', () => {
    const req = {
      isAuthenticated: jest.fn(() => false),
    };
    isNotLoggedIn(req, res, next);
    expect(next).toBeCalledTimes(1);
  });
});
```



4. 라우터 테스트 위해 분리하기

» 라우터도 미들웨어이므로 분리해서 테스트 가능

controllers/user.js

```
const User = require('../models/user');

exports.addFollowing = async (req, res, next) => {
  try {
    const user = await User.findOne({ where: { id: req.user.id } });
    if (user) {
      await user.addFollowing(parseInt(req.params.id, 10));
      res.send('success');
    } else {
      res.status(404).send('no user');
    }
  } catch (error) {
    console.error(error);
    next(error);
  }
};
```

routes/user.js

```
const express = require('express');

const { isLoggedIn } = require('./middlewares');
const { addFollowing } = require('../controllers/user');

const router = express.Router();

router.post('/:id/follow', isLoggedIn, addFollowing);

module.exports = router;
```



5. 라우터 테스트

» Controllers/user.test.js 작성하기

controllers/user.test.js

```
const { addFollowing } = require('./user');

describe('addFollowing', () => {
  const req = {
    user: { id: 1 },
    params: { id: 2 },
  };
  const res = {
    status: jest.fn(() => res),
    send: jest.fn(),
  };
  const next = jest.fn();

  test('사용자를 찾아 팔로잉을 추가하고 success를 응답해야 함', async () => {
    await addFollowing(req, res, next);
    expect(res.send).toBeCalledWith('success');
  });

  test('사용자를 못 찾으면 res.status(404).send(no user)를 호출함', async () => {
    await addFollowing(req, res, next);
    expect(res.status).toBeCalledWith(404);
    expect(res.send).toBeCalledWith('no user');
  });

  test('DB에서 에러가 발생하면 next(error) 호출함', async () => {
    const error = '테스트용 에러';
    await addFollowing(req, res, next);
    expect(next).toBeCalledWith(error);
  });
});
```

콘솔

```
$ npm test
> nodebird@0.0.1 test C:\Users\zerocho\nodebird
> jest
```

PASS routes/middlewares.test.js

FAIL controllers/user.test.js

● Console

```
console.error controllers/user.js:10
  TypeError: Cannot convert undefined or null to object
console.error controllers/user.js:10
  TypeError: Cannot convert undefined or null to object
console.error controllers/user.js:10
  TypeError: Cannot convert undefined or null to object
```

● addFollowing > 사용자를 찾아 팔로잉을 추가하고 success를 응답해야 함

```
expect(jest.fn()).toBeCalledWith(...expected)
```

Expected: "success"

Number of calls: 0

```
13 |   test('사용자를 찾아 팔로잉을 추가하고 success를 응답해야 함', async () => {
14 |     await addFollowing(req, res, next);
> 15 |     expect(res.send).toBeCalledWith('success');
    |                                     ^
16 |   });
17 |
18 |   test('사용자를 못 찾으면 res.status(404).send(no user)를 호출함', async () => {
```

at Object.<anonymous> (controllers/user.test.js:15:22)



6. DB 모킹하기

» Jest를 사용해 모듈 모킹 가능(jest.mock)

- 메서드에 mockReturnValue 메서드가 추가되어 리턴값 모킹 가능

controllers/user.test.js

```
jest.mock('../models/user');
const User = require('../models/user');
const { addFollowing } = require('./user');

describe('addFollowing', () => {
  const req = {
    user: { id: 1 },
    params: { id: 2 },
  };
  const res = {
    status: jest.fn(() => res),
    send: jest.fn(),
  };
  const next = jest.fn();

  test('사용자를 찾아 팔로잉을 추가하고 success를 응답해야 함', async () => {
    User.findOne.mockReturnValue(Promise.resolve({
      addFollowing(id) {
        return Promise.resolve(true);
      }
    }));
    await addFollowing(req, res, next);
    expect(res.send).toHaveBeenCalledWith('success');
  });

  test('사용자를 못 찾으면 res.status(404).send(no user)를 호출함', async () => {
    User.findOne.mockReturnValue(null);
    await addFollowing(req, res, next);
    expect(res.status).toHaveBeenCalledWith(404);
    expect(res.send).toHaveBeenCalledWith('no user');
  });
});
```

콘솔

```
$ npm test
> nodebird@0.0.1 test C:\Users\zerocho\nodebird
> jest
```

PASS routes/middlewares.test.js

PASS controllers/user.test.js

● Console

console.error controllers/user.js:13

테스트용 에러

Test Suites: 2 passed, 2 total

Tests: 6 passed, 6 total

Snapshots: 0 total

Time: 3.562s

Ran all test suites.

11.3 테스트 커버리지



1. 테스트 커버리지란?

» 전체 코드 중에서 테스트되고 있는 코드의 비율

- 테스트되지 않는 코드의 위치도 알려줌
- `jest --coverage`
- Stmts: 구문
- Branch: 분기점
- Funcs: 함수
- Lines: 줄 수

package.json

```
{
  "name": "nodebird",
  "version": "0.0.1",
  "description": "익스프레스로 만드는 SNS 서비스",
  "main": "app.js",
  "scripts": {
    "start": "nodemon app",
    "test": "jest",
    "coverage": "jest --coverage"
  },
}
```

콘솔

```
$ npm run coverage
> nodebird@0.0.1 coverage C:\Users\zerocho\nodebird
> jest --coverage
```

PASS routes/middlewares.test.js

PASS controllers/user.test.js

● Console

console.error controllers/user.js:13

사용자 못 찾을

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	83.33	83.33	60	83.33	
controllers	100	100	100	100	
user.js	100	100	100	100	
models	33.33	100	0	33.33	
user.js	33.33	100	0	33.33	5,41,42,47
routes	100	100	100	100	
middlewares.js	100	100	100	100	

Test Suites: 2 passed, 2 total

Tests: 6 passed, 6 total

Snapshots: 0 total

Time: 5.499s

Ran all test suites.



2. 테스트 커버리지 올리기

» models/users.js의 5, 41, 42, 47줄 확인

models/user.js

```
const Sequelize = require('sequelize');

module.exports = class User extends Sequelize.Model {
  static init(sequelize) {
    return super.init({
      email: {
        type: Sequelize.STRING(40),
        allowNull: true,
        unique: true,

static associate(db) {
  db.User.hasMany(db.Post);
  db.User.belongsToMany(db.User, {
    db.User.belongsToMany(db.User, {
      foreignKey: 'followerId',
      as: 'Followings',
      through: 'Follow',
    });
  });
}
```



3. 테스트 커버리지 올리기

» models/users.test.js 작성

models/user.test.js

```
const Sequelize = require('sequelize');
const User = require('./user');
const config = require('../config/config')['test'];
const sequelize = new Sequelize(
  config.database, config.username, config.password, config,
);
```

```
describe('User 모델', () => {
  test('static init 메서드 호출', () => {
    expect(User.init(sequelize)).toBe(User);
  });
  test('static associate 메서드 호출', () => {
    const db = {
      User: {
        hasMany: jest.fn(),
        belongsToMany: jest.fn(),
      },
      Post: {},
    };
    User.associate(db);
    expect(db.User.hasMany).toHaveBeenCalledTimes(1);
    expect(db.User.belongsToMany).toHaveBeenCalledTimes(2);
  });
});
```

콘솔

```
$ npm test
> nodebird@0.0.1 test C:\Users\zerocho\nodebird
> jest
```

(성공 메시지 생략)

콘솔

```
$ npm run coverage
(표 이외 부분 생략)
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	100	100	100	100	
controllers	100	100	100	100	
user.js	100	100	100	100	
models	100	100	100	100	
user.js	100	100	100	100	
routes	100	100	100	100	
middlewares.js	100	100	100	100	

4. 테스트 커버리지 주의점

» 모든 코드가 테스트되지 않는데도 커버리지가 100%임

- 테스트 커버리지를 맹신할 필요가 없음
- 커버리지를 높이는 것이 의미는 있지만 높이는 데 너무 집착할 필요는 없음
- 필요한 부분 위주로 올바르게 테스트하는 것이 좋음

11.4 통합 테스트



1. 통합 테스트 해보기

» 라우터 하나를 통째로 테스트해 봄(여러 개의 미들웨어, 모듈을 한 번에 테스트).

- app.js 분리하기

- app.js

```
...
app.use((err, req, res, next) => {
  console.error(err);
  res.locals.message = err.message;
  res.locals.error = process.env.NODE_ENV !== 'production' ? err : {};
  res.status(err.status || 500);
  res.render('error');
});
```

```
module.exports = app;
```

server.js

```
const app = require('./app');

app.listen(app.get('port'), () => {
  console.log(app.get('port'), '번 포트에서 대기중');
});
```

package.json

```
{
  "name": "nodebird",
  "version": "0.0.1",
  "description": "익스프레스로 만드는 SNS 서비스",
  "main": "app.js",
  "scripts": {
    "start": "nodemon server",
    "test": "jest",
    ...
  }
}
```



2. 테스트용 DB 설정하기

» 개발/배포용 DB랑 별도로 설정하는 것이 좋음

- config/config.json의 test 속성

config/config.json

```
{
  "development": {
    "username": "root",
    "password": "nodejsbook",
    "database": "nodebird",
    "host": "127.0.0.1",
    "dialect": "mysql"
  },
  "test": {
    "username": "root",
    "password": "비밀번호",
    "database": "nodebird_test",
    "host": "127.0.0.1",
    "dialect": "mysql"
  },
  ...
}
```

콘솔

```
$ npx sequelize db:create --env test
Sequelize CLI [Node: 14.0.0, CLI: 6.1.0, ORM: 6.2.3]

Loaded configuration file "config\config.json".
Using environment "test".
Database nodebird_test created.
```



3. 라우터 테스트

» routes/auth.test.js 작성

- beforeAll: 모든 테스트 전에 실행
- request(app).post(주소)로 요청
- send로 data 전송

콘솔

```
$ npm test
(다른 테스트 로그 생략)
FAIL routes/auth.test.js
  ● POST /login > 로그인 수행

    expected "Location" of "/", got "/?loginError..."
(생략)
POST /auth/login 302 87.548 ms - 3296
Test Suites: 1 failed, 3 passed, 4 total
Tests:      1 failed, 7 passed, 8 total
Snapshots:  0 total
Time:       7.175s
Ran all test suites.
A worker process has failed to exit gracefully and has been force exited. This is likely caused by tests leaking due to improper teardown. Try running with --runInBand --detectOpenHandles to find leaks.
npm ERR! Test failed. See above for more details.
```

routes/auth.test.js

```
const request = require('supertest');
const { sequelize } = require('../models');
const app = require('../app');

beforeAll(async () => {
  await sequelize.sync();
});

describe('POST /login', () => {
  test('로그인 수행', async (done) => {
    request(app)
      .post('/auth/login')
      .send({
        email: 'zerohch@gmail.com',
        password: 'nodejsbook',
      })
      .expect('Location', '/')
      .expect(302, done);
  });
});
```




4. 회원 정보부터 만들기

» routes/auth.test.js 수정

- 테스트 실행하면 성공함
- 재차 실행하면 실패함

콘솔

```
$ npm test
(다른 테스트 및 Console 생략)
POST /auth/login 302 242.585 ms - 23
POST /auth/join 302 10.592 ms - 115

Test Suites: 4 passed, 4 total
Tests: 11 passed, 11 total
Snapshots: 0 total
Time: 6.253s
Ran all test suites.
A worker process has failed to exit gracefully and has been force exited. This is likely caused by tests leaking due to improper teardown. Try running with --runInBand --detectOpenHandles to find leaks.
```

콘솔

```
$ npm test
(다른 테스트 및 Console 생략)
● POST /join > 로그인 안 했으면 가입

    expected "Location" of "/", got "/join?error=exist"
(생략)
POST /auth/login 302 242.168 ms - 23
POST /auth/join 302 9.900 ms - 115

Test Suites: 1 failed, 3 passed, 3 total
Tests: 1 failed, 10 passed, 11 total
Snapshots: 0 total
Time: 6.179s
Ran all test suites.
A worker process has failed to exit gracefully and has been force exited. This is likely caused by tests leaking due to improper teardown. Try running with --runInBand --detectOpenHandles to find leaks.
npm ERR! Test failed. See above for more details.
```

```
describe('POST /join', () => {
  test('로그인 안 했으면 가입', (done) => {
    request(app)
      .post('/auth/join')
      .send({
        email: 'zerohch0@gmail.com',
        nick: 'zerocho',
        password: 'nodejsbook',
      })
      .expect('Location', '/')
      .expect(302, done);
  });
});

describe('POST /join', () => {
  const agent = request.agent(app);
  beforeEach((done) => {
    agent
      .post('/auth/login')
      .send({
        email: 'zerohch0@gmail.com',
        password: 'nodejsbook',
      })
      .end(done);
  });

  test('이미 로그인했으면 redirect /', (done) => {
    const message = encodeURIComponent('로그인한 상태입니다.');
```



5. afterAll로 정리하기

» routes/auth.test.js 수정

- afterAll은 테스트가 종료된 후에 실행됨
- DB 초기화하기

auth.test.js

```
...  
test('이미 로그인했으면 redirect /', (done) => {  
  const message = encodeURIComponent('로그인한 상태입니다.');
```

```
  agent  
    .post('/auth/join')  
    .send({  
      email: 'zerohch@gmail.com',  
      nick: 'zerocho1',  
      password: 'nodejsbook',  
    })  
    .expect('Location', `/?error=${message}`)  
    .expect(302, done);  
});  
  
afterAll(async () => {  
  await sequelize.sync({ force: true });  
});
```



6. 로그아웃 테스트하기

» routes/auth.test.js 수정

```
describe('GET /logout', () => {
  test('로그인되어 있지 않으면 403', async (done) => {
    request(app)
      .get('/auth/logout')
      .expect(403, done);
  });

  const agent = request.agent(app);
  beforeEach((done) => {
    agent
      .post('/auth/login')
      .send({
        email: 'zerohch0@gmail.com',
        password: 'nodejsbook',
      })
      .end(done);
  });

  test('로그아웃 수행', async (done) => {
    const message = encodeURIComponent('비밀번호가 일치하지 않습니다. ');
    agent
      .get('/auth/logout')
      .expect('Location', '/')
      .expect(302, done);
  });
});
```

11.5 부하 테스트



1. 부하 테스트란?

» 서버가 얼마만큼의 요청을 견딜 수 있는지 테스트

- 서버가 몇 명의 동시 접속자를 수용할 수 있는지 예측하기 매우 어려움
- 실제 서비스 중이 아니라 개발 중일 때는 더 어려움
- 코드에 문제가 없더라도 서버 하드웨어 때문에 서비스가 중단될 수 있음(메모리 부족 문제 등)
- 부하 테스트를 통해 미리 예측할 수 있음



콘솔

```
$ npm i -D artillery
```

```
$ npm start
```



2. Artillery 사용하기

» 새 콘솔에서 다음 명령어 입력

- Count 옵션은 가상의 사용자 수
- N 옵션은 횟수
- 100명의 사용자가 50번씩 요청을 보내는

» 결과 보고서

- 사용자 생성(scenarios launched)
- 테스트 성공(scenarios completed)
- 요청 성공 횟수(requests completed)
- 초당 요청 처리 횟수(RPS sent)
- 응답 지연 속도(Request latency)
- Min: 최소, Max: 최대, median: 중앙값
- P95: 하위 95%, P99: 하위 99%
 - 하위는 속도 순서를 말함
- Median과 P95가 많이 차이 나지 않는 게 좋음

콘솔

```
$ npx artillery quick --count 100 -n 50 http://localhost:8001
(중간 로그 생략)
All virtual users finished
Summary report @ 22:06:27(+0900) 2020-02-26
Scenarios launched: 100
Scenarios completed: 100
Requests completed: 5000
RPS sent: 474.38
Request latency:
  min: 9.1
  max: 303.1
  median: 176.2
  p95: 252.2
  p99: 276.7
Scenario counts:
  0: 100 (100%)
Codes:
  200: 5000
```



3. 여러 페이지 요청 시나리오

» loadtest.json에 사용자의 행동 흐름 작성 가능

- target: 요청 도메인
 - Phases에서 duration: 몇 초 동안(60초)
 - arrivalRate: 매 초 몇 명(30명)
 - flow: 사용자의 이동
 - get, post 등의 메서드를 나타냄
 - url은 이동한 url
 - json은 서버로 전송한 데이터
-
- 현재 GET /, POST /auth/login, GET /hashtag 순

loadtest.json

```
{
  "config": {
    "target": "http://localhost:8001",
    "phases": [
      {
        "duration": 60,
        "arrivalRate": 30
      }
    ]
  },
  "scenarios": [
    {
      "flow": [
        {
          "get": {
            "url": "/"
          }
        }, {
          "post": {
            "url": "/auth/login",
            "json": {
              "email": "zerohch@gmail.com",
              "password": "nodejsbook"
            }
          }
        }, {
          "get": {
            "url": "/hashtag?hashtag=nodebird"
          }
        }
      ]
    }
  ]
}
```



4. 여러 페이지 요청 시나리오(실행)

» 문제점 발견

- 요청 후반부가 될 수록 응답 시간이 길어짐
- 첫 응답은 4.7밀리초, 마지막 응답은 51초
- 5400개의 요청은 200 응답코드, 1800개는 302
- 서버가 지금 정도의 요청을 감당하지 못함
- 서버 사양을 업그레이드하거나, 여러 개 두거나
- 코드를 더 효율적으로 개선하는 방법 등.
- 현재는 싱글코어만 사용하므로, 클러스터링 기법 도입을 시도해볼만 함
- arrivalRate를 줄이거나 늘려서 어느 정도 수용 가능한지 체크해보는 것이 좋음
- 여러 번 테스트하여 평균치를 내보는 게 좋음

콘솔

```
$ npx artillery run loadtest.json
```

(중간 로그 생략)

All virtual users finished

Summary report @ 23:09:36(+0900) 2020-02-26

Scenarios launched: 1800

Scenarios completed: 1800

Requests completed: 7200

RPS sent: 50.2

Request latency:

min: 4.7

max: 51806.1

median: 19150.8

p95: 45585.6

p99: 49574.6

Scenario counts:

0: 1800 (100%)

Codes:

200: 5400

302: 1800



5. 테스트 범위

» 어떤 것을 테스트하고 어떤 것을 테스트 안 할 지 고민됨.

- 자신이 짠 코드는 최대한 많이 테스트하기
 - npm을 통해 설치한 패키지는 테스트하지 않음(그걸 만든 사람의 몫임)
 - 우리는 그 패키지/라이브러리를 사용하는 부분만 테스트
 - 테스트하기 어려운 패키지는 모킹
 - 모킹해서 통과하더라도 실제 상황에서는 에러날 수 있음을 염두에 두어야 함
-
- 시스템 테스트: QA처럼 테스트 목록을 두고 체크해나가면서 진행하는 테스트
 - 인수 테스트: 알파 테스트/베타 테스트처럼 특정 사용자 집단이 실제로 테스트
 - 다양한 종류의 테스트를 주기적으로 수행해 서비스를 안정적으로 유지하는 게 좋음